TP noté : Pipeline DevSecOps avec GitHub Actions

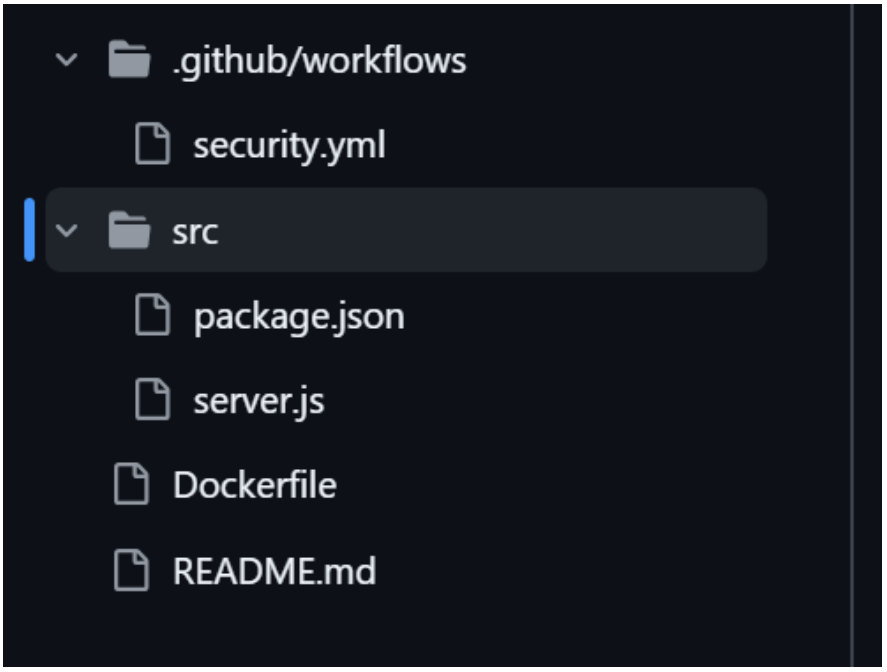
Yuna GILBERT - M1 CSAI2
Prérequis : Docker, Git, Linux

Objectifs :

- Mettre en place un pipeline CI/CD sécurisé
- Détecter automatiquement les vulnérabilités (SAST, SCA, DAST)
- Corriger les failles de sécurité courantes
- Comprendre le DevSecOps en pratique

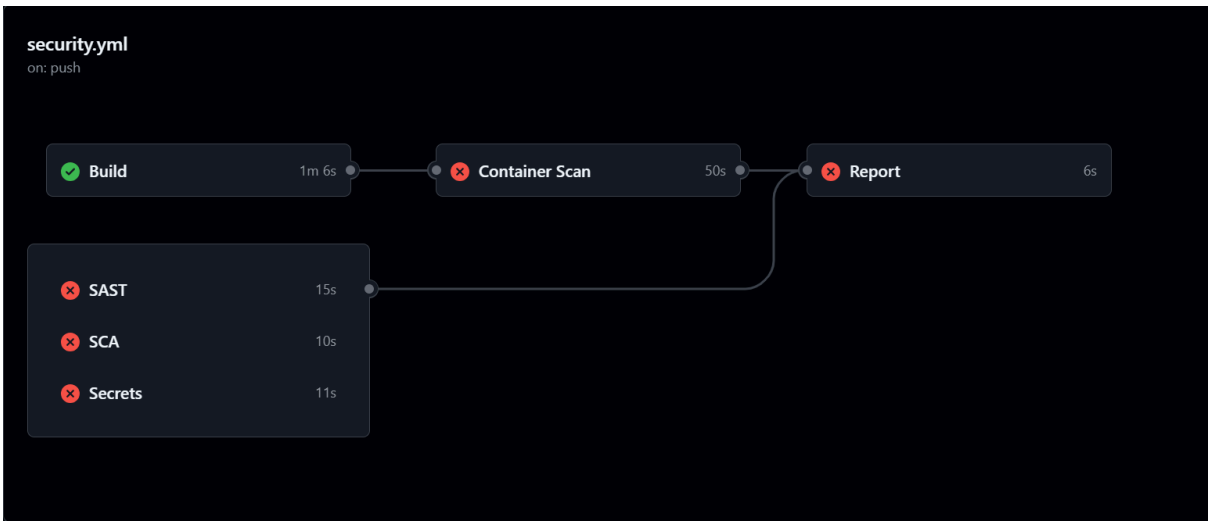
Section 1 & 2, mise en place de l'application :

La section 1 & 2 du TP noté nous permet de mettre en place cette application Node.js qui est vulnérable, si notre github on se retrouve avec cette répartition là :



Section 3, les vulnérabilités :

Après l'exécution du commit, on peut voir dans l'onglet Actions de github ceci :



Ici ce sont les vulnérabilités trouvées avant notre correction.

On voit ici que mon image a été bien construite, et que mon application s'est compilée, mais ensuite on voit ce qui échoue car le pipeline détecte des failles :

- SAST (semgrep) détecte :
  secrets hardcodés

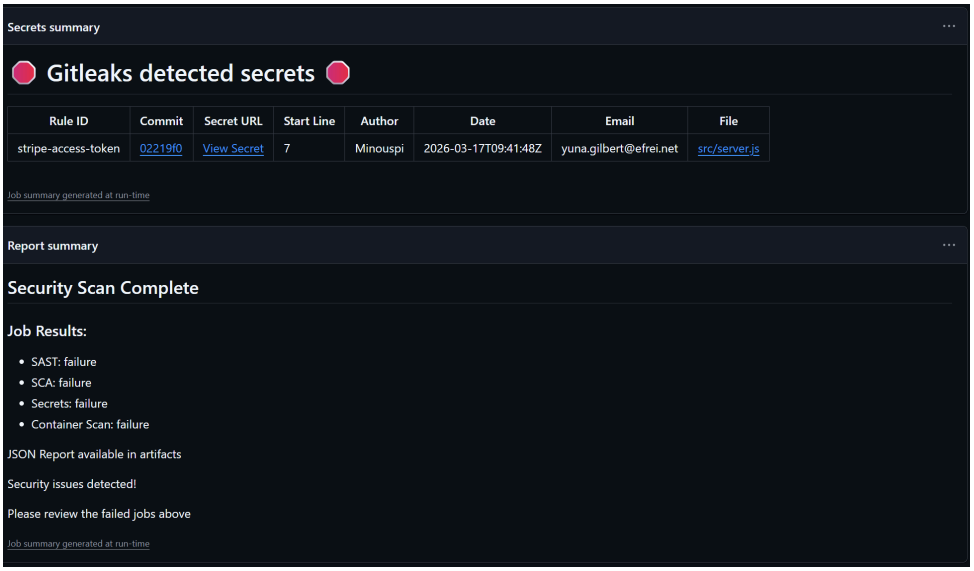
```
const STRIPE_SECRET_KEY = "sk_live_51Hqp9K2eZvKYlo2C8xO3n4y5z6a7b8c9d0e1f2g3h4i5j";
```

  Pas de validation (identification utilisateur)
- SCA (npm audit) détecte :
  Dépendance vulnérable (CVE)
- Secrets (Gitleak) détecte :
  clé API dans le repo

```
const SENDGRID_API_KEY = "SG.nExT2-QRDzJcEV39HqCxTg.KnLmOpQrStUvWxYz1234567890aBcDeF";
```

- Container Scan (Trivy) détecte :
  vulnérabilités dans l'image Docker

Les rapports :



Secrets summary

Gitleaks detected secrets

Rule ID	Commit	Secret URL	Start Line	Author	Date	Email	File
stripe-access-token	02219f0	View Secret	7	Minouspi	2026-03-17T09:41:48Z	yuna.gilbert@efrei.net	src/server.js

Job summary generated at run-time

Report summary

Security Scan Complete

Job Results:

- SAST: failure
- SCA: failure
- Secrets: failure
- Container Scan: failure

JSON Report available in artifacts

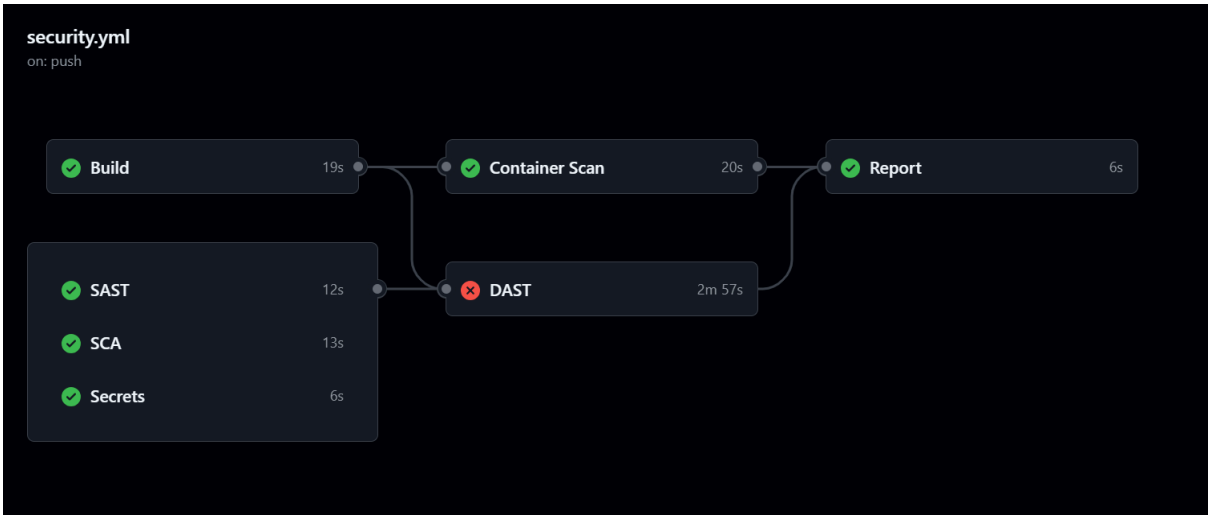
Security issues detected!

Please review the failed jobs above

Job summary generated at run-time

Section 4, les corrections apportées :

Après la correction de package.json, server.js et Dockerfile :



package.json :

Code Blame 12 lines (12 loc) · 227 Bytes

```
{
 "name": "secure-app",
 "version": "2.0.0",
 "dependencies": {
 "express": "^4.18.2",
 "jsonwebtoken": "^9.0.2",
 "helmet": "^7.1.0",
 "express-rate-limit": "^7.1.0",
 "express-validator": "^7.0.1",
 "dotenv": "^16.3.1"
 }
}
```

server.js :

On a retiré les clés en clair du code, et on a créé des secrets sur Github.

Code Blame 70 lines (59 loc) · 1.76 KB

```
require('dotenv').config();
const express = require('express');
const jwt = require('jsonwebtoken');
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');
const { body, validationResult } = require('express-validator');

const app = express();

```

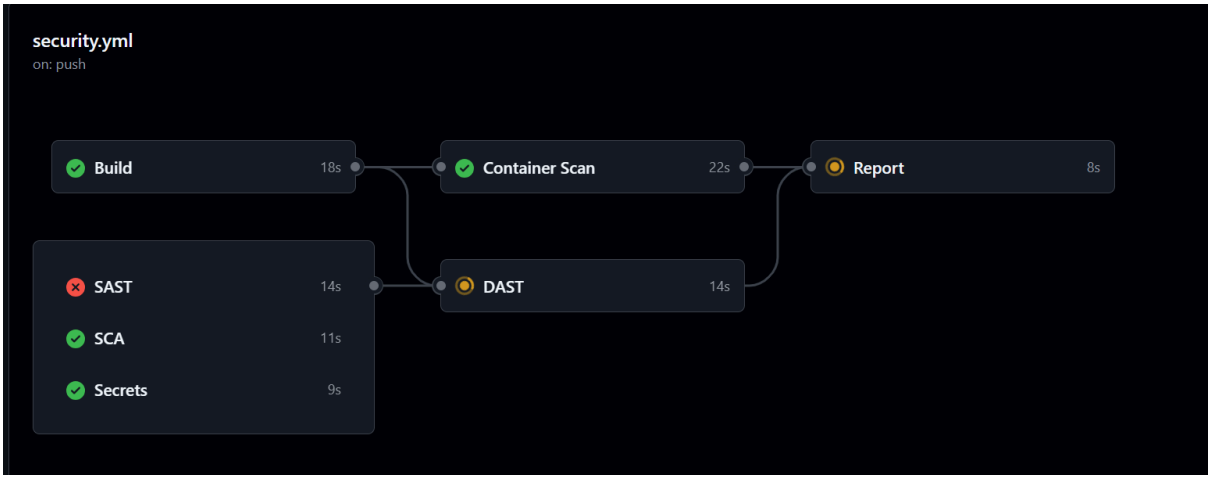
Dockerfile :

```
root@debian:/home/yg/devsecops-lab-yg# cat Dockerfile
# Image Alpine sécurisée avec version pin
FROM node:20.11.1-alpine3.19

WORKDIR /app

# Copie des dépendances d'abord (cache)
COPY src/package*.json ./
RUN npm ci --only=production && npm cache clean --force

COPY src/ ./

# Utilisateur non-root
RUN addgroup -g 1001 -S nodejs && \
    adduser -S nodejs -u 1001 && \
    chown -R nodejs:nodejs /app

USER nodejs

EXPOSE 3000

# Healthcheck
HEALTHCHECK --interval=30s --timeout=3s \
    CMD node -e "require('http').get('http://localhost:3000/health', (r) => process.exit(r.statusCode === 200 ? 0 : 1))"

CMD ["node", "server.js"]
```

Exercices pratiques :

Exercice 1 :

Injection SQL dans le code server.js :
détection :

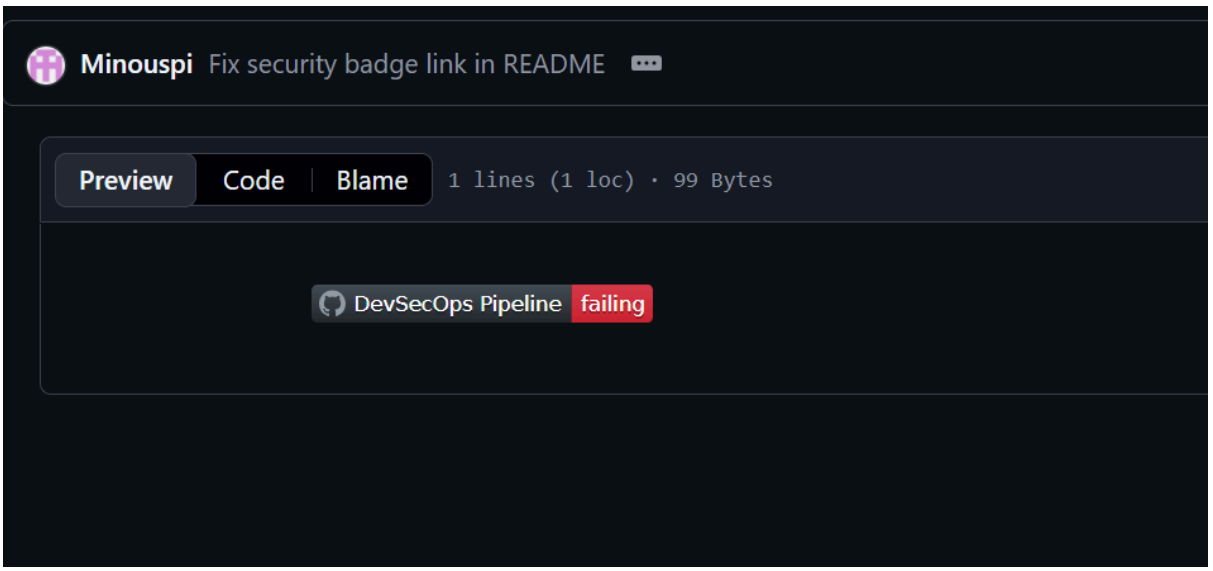


L'injection :

```
const sqlite3 = require('sqlite3'); // important
const db = new sqlite3.Database(':memory:');

app.get('/api/user', (req, res) => {
  const userId = req.query.id;

  // ❌ SQL Injection claire pour Semgrep
  const query = "SELECT * FROM users WHERE id = " + userId;

  db.all(query, [], (err, rows) => {   // 👈 clé pour détection
    if (err) {
      return res.status(500).json({ error: err.message });
    }
    res.json({ rows });
  });
});
```

Exercice 2 :

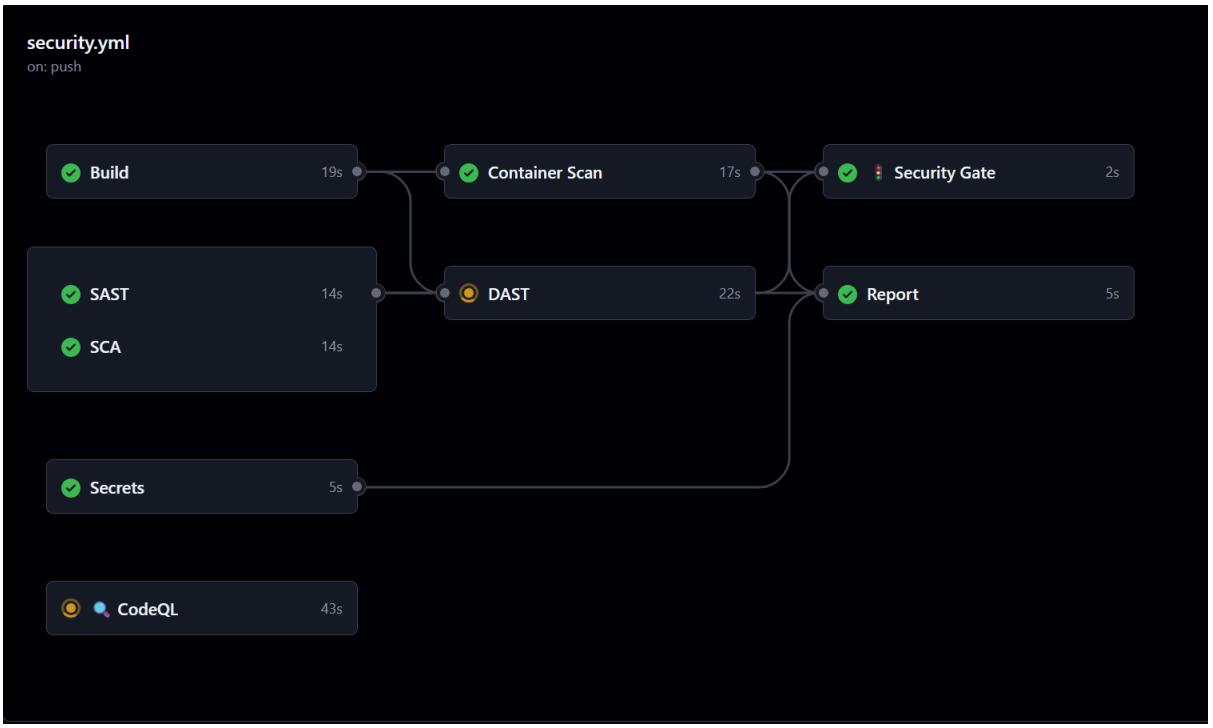
Badge de sécurité



Il est en failing, car la partie DAST échoue à cause d'un problème de zaplink.

Exercice 3 et 4 :

Ajout de CodeQL et d'un job qui bloque les merges si les vulnérabilités sont critiques



On voit qu'ils sont bien intégrés dans le workflow.

Synthèse :

Evaluation de la criticité des vulnérabilités trouvées après la section 1 et 2 :

Niveau	Signification
Critical	Impact très grave (exécution de code, fuite massive)
High	Exploitation facile avec impact important
Medium	Risque modéré
Low	Faible impact

Pour nos vulnérabilités trouvées :

Type de vulnérabilité	Outil	Sévérité
Command Injection	Semgrep	Critical
Dépendances vulnérables	npm audit	High
Secrets exposés	Gitleaks	High
Vulnérabilités image Docker	Trivy	Critical

Les leçons apprises :

Au cours de ce projet, j'ai appris que la sécurité doit être prise en compte dès le début du développement et non ajoutée à la fin. J'ai également compris que les entrées utilisateur doivent toujours être validées afin d'éviter les attaques comme les injections. Ce projet m'a montré que les dépendances peuvent contenir des vulnérabilités, ce qui rend leur mise à jour régulière indispensable. J'ai aussi appris qu'il ne faut jamais stocker de secrets directement dans le code source, mais utiliser des variables d'environnement sécurisées. Enfin, j'ai constaté que l'utilisation d'un pipeline DevSecOps permet de détecter automatiquement les vulnérabilités et d'améliorer la sécurité globale de l'application.